Réflexion O76
Analyse des sources et justification de la conception finale

Construction O76

30 septembre 2026

Table des matières

1 Périmètre de l'analyse **1**

2 Première source : théorie DDD **1**

3 Deuxième source : formalisme de l'observation 0 **2**

4 Héritage des générations O69–O75 **2**

5 Le registre d'erreurs comme matériau de conception **3**

6 Pourquoi une architecture de certificat plutôt qu'un modèle de langage **3**

7 Réflexion finale sur l'architecture O76 **3**
- 7.1 Séparation observation / vérité 3
- 7.2 Séparation génération / vérification 3
- 7.3 Séparation mémoire / oracle 4
- 7.4 Séparation budget / impossibilité 4
- 7.5 Séparation expérimentation / démonstration 4

8 Ce qui est véritablement nouveau dans O76 **4**

9 Réflexion sur le test décisif et la correction O76-1.1 **4**

10 Conclusion **5**

1 Périmètre de l'analyse

Les deux archives fournies ont été traitées récursivement, y compris leurs archives imbriquées. Les hashes SHA-256 des deux archives mères examinées sont respectivement `7e1874173e824a9ed5480882573f94ebd61` et `87ac29533923e4e7101980637920622de2af764a29c78f990e6afb399197010b`. L'inventaire a porté sur 732 fichiers extraits, dont 139 Python, 65 TeX, 59 Markdown, 39 TXT, 310 JSON, 2 JSONL, 16 PDF, 9 checkpoints PyTorch et 32 fichiers gzip, plus des fichiers auxiliaires. Les empreintes de contenu montrent 455 contenus distincts et 144 groupes de doublons dans le corpus récursivement extrait. Les deux archives mères contiennent respectivement 119 et 33 membres de fichiers non-répertoires.

Tous les fichiers Python du corpus ont passé la compilation syntaxique. Les 16 PDF ont été lus par conversion locale en texte avec succès. Les 304 JSON valides, 2 JSONL et 32 gzip ont été contrôlés. Six JSON étaient vides et illisibles : quatre sorties `hard5` et deux sorties `stress4` ; le registre historique E38 les décrit déjà comme des artefacts de coupure du harness. Ils ont donc été considérés comme des absences de résultat, pas comme des preuves négatives.

2 Première source : théorie DDD

Le document central `DDD_Base_theorie_des_empreintes_complete_order_5.tex` comporte 3922 lignes et 217783 octets. Le texte contient 27 environnements de théorème, 45 définitions, 49 propositions et 18 corollaires selon le décompte structurel du fichier ; il contient également des axiomes et une conjecture. Son rôle n'est pas de fournir un solveur arithmétique prêt à l'emploi, mais de donner une théorie de l'observation, de la distinction et de la transformation.

Les éléments que je considère structurellement décisifs pour O76 sont les suivants.

Indiscernabilité et réduction. Si une observation fine implique une observation plus grossière, la trajectoire peut changer de représentation sans changer les distinctions déjà préservées. Cette idée justifie un routeur par empreinte : la réduction porte sur la forme observable, jamais sur la vérité du résultat.

Fermeture observationnelle. La source lie la factorisation et la fermeture à une compatibilité des transformations. O76 l'utilise pour imposer qu'un changement de route soit une transformation définie et enregistrée.

Double minimum. La loi de type ultramétrique $\Theta_P(\{x,z\}) \geq \min(\Theta_P(\{x,y\}), \Theta_P(\{y,z\}))$ est mathématiquement intéressante, mais elle ne peut pas être utilisée comme raccourci pour prouver une équation arbitraire. Elle est donc conservée dans le méta-modèle, pas comme oracle.

Trajectoire, déficit, témoin. Cette partie inspire le couple « trajectoire de résolution / certificat ou contre-certificat ». Une trajectoire qui échoue doit laisser une trace de frontière, pas être transformée silencieusement en impossibilité.

Recherche récursive. La complétude conditionnelle de la recherche par approfondissement, lorsque la profondeur est finie, le facteur de branchement fini et l'espace décidabilité disponible, justifie un portefeuille itératif. O76 ne transforme toutefois pas ces conditions en une promesse de complétude globale sur toutes les mathématiques.

3 Deuxième source : formalisme de l'observation 0

Le document `th_emp_observation_formalisme_0.tex` comporte 496 lignes et 34654 octets, avec 19 définitions, 10 propositions et 1 corollaire dans son texte. Sa nouveauté la plus importante pour O76 est la séparation entre existence ontologique $E_o(x)$ et présence relative $E_r(x \mid s,t)$, ainsi que le traitement de l'observation comme distinction fonctionnelle dépendante du contexte.

Observation locale. Une différence n'est une observation que si elle dépend d'un objet ou d'un état discriminé. Ceci mène à une règle de conception explicite : le feedback d'un solveur n'est pas automatiquement une observation utile ; il faut que la sortie soit une distinction mathématique portant sur le problème.

Mémoire ontologique et mémoire évolutive. Le document propose M_o comme fondement et M_e comme mémoire de l'évolution. O76 implémente seulement la partie qui est calculable sans prétention métaphysique : les concepts structurants sont déclarés ; les événements de trajectoire et les certificats vérifiés sont mémorisés par hash.

Boucle fermée. La condition « la sortie est réintroduite comme entrée avec dépendance causale effective » est directement adaptée au pipeline : une empreinte influence la trajectoire, le certificat influence la mémoire, la mémoire influence une nouvelle sélection. Le modèle obtient ainsi une forme d'émergence fonctionnelle, observable dans les journaux d'exécution.

4 Héritage des générations O69–O75

Les archives montrent un déplacement progressif d'un modèle hybride et orienté langage vers un runtime mathématique exact. Les versions récentes ajoutent Buchberger, Sturm, matrices rationnelles, CRT, Diophantien, Pell, checkpoints et séparation du vérificateur. Les expériences Riemann ont également clarifié une frontière essentielle : une vérification numérique sur une plage finie ne constitue pas une preuve universelle.

Le runtime O75 le plus abouti possédait déjà une base sérieuse d'arithmétique rationnelle et de certificat. Mais quatre faiblesses restaient structurantes.

1. Une base de Gröbner complète pouvait encore être confondue avec une preuve structurelle. O76 attache des représentations exactes à chaque élément et livre une combinaison finale dans les générateurs initiaux.
2. Certains certificats linéaires pouvaient être décrits comme valides avec trop peu de rejeu. O76 recalcule la RREF et compare les expressions paramétriques aux lignes pivots.
3. Les checkpoints utilisaient historiquement des conversions génériques comme `default=str` ; O76 sérialise les polynômes sous forme structurée, et vérifie le hash canonique du problème.
4. Les branches analytiques RH, Li ou zêta pouvaient produire des informations utiles mais non universelles. O76 les exclut du chemin de preuve.

5 Le registre d'erreurs comme matériau de conception

Le fichier `erreur_model_experience_2_v2.txt` fournit la mémoire la plus exploitable. Les erreurs E23–E30 ont une correspondance directe dans le noyau O76 :

Erreur	Leçon	Garde O76
E23	partie square-free mal calculée	division exacte par $\gcd(p, p')$
E24	projection univariée incorrecte	conversion explicite vers un polynôme à une variable
E25	racines négatives / zéro perdues	extraction itérative et candidats $\pm p/q$
E26	bornes d'inégalité mal représentées	séparation entre racines exactes et intervalles
E27	base Gröbner présentée comme preuve	certificat d'identité dans l'idéal
E28	checkpoint non sérialisable	`poly_payload` et état canonique
E29	mauvais déballage terminal	résultat terminal vérifié, pas conteneur brut
E30	cible entière passée à un parseur d'expression	séparation relationnelle obligatoire
E31–E35	reprise, mutation, budget, empreinte	hash de problème, deadlines et checkpoints structurels
E38–E42	harness/bench/checker	stdout non autoritaire, checker séparé, frontières explicites
E43–E58	illusion de preuve RH finie	aucune statistique numérique n'est autorité de preuve

6 Pourquoi une architecture de certificat plutôt qu'un modèle de langage

Le terme « modèle » ne force pas l'emploi d'un réseau neuronal. Pour un système dédié aux mathématiques et exigeant des démonstrations rigoureuses, la propriété la plus importante est la

capacité à distinguer une proposition plausible d'une proposition certifiée. Un modèle local purement neuronal peut être ajouté plus tard comme proposeur de stratégies ou de lemmes candidats, mais il serait alors un composant non autoritaire.

O76 prend donc le choix inverse : l'émergence vient de la boucle de raisonnement structurée, et non d'une réponse statistique. Un futur proposeur peut générer g, mais seule la construction d'un certificat exact suivie de son rejeu transforme g en connaissance du système.

7 Réflexion finale sur l'architecture O76

La conception finale repose sur cinq séparations.

7.1 Séparation observation / vérité

L'empreinte est une observation de forme. Le vérificateur est l'autorité de vérité computationnelle. La théorie des empreintes ne se substitue jamais au calcul exact.

7.2 Séparation génération / vérification

Le solveur génère. Le vérificateur rejoue. Cette asymétrie limite fortement le risque qu'une même erreur interne se masque elle-même. Elle ne constitue cependant pas une preuve formelle du code : cette précision reste dans le contrat de confiance.

7.3 Séparation mémoire / oracle

La mémoire ne contient ni réponse cible cachée ni table de benchmark. Elle contient uniquement des empreintes, des hashes, des provenances et des certificats qui doivent de toute façon être rejoués.

7.4 Séparation budget / impossibilité

Un budget épuisé produit UNRESOLVED. La ressource est relative au run, exactement dans l'esprit du traitement relationnel du fini et de l'infini présent dans les documents `infini_vs_fini`. Cela interdit l'abus logique consistant à lire « je n'ai pas fini » comme « aucun théorème n'existe ».

7.5 Séparation expérimentation / démonstration

Les benchmarks, stress, sondes et résultats historiques restent utiles pour mesurer la couverture et identifier les défauts. Mais ils sont produits hors du chemin logique de décision et ne peuvent pas faire évoluer automatiquement la règle de preuve.

8 Ce qui est véritablement nouveau dans O76

La principale nouveauté n'est pas le nombre de solveurs. Elle est la combinaison suivante :

empreinte relationnelle → trajectoire adaptative → représentation de preuve → rejeu indépendant → mémoire év

Dans le cas Gröbner, cette architecture devient particulièrement concrète grâce au suivi des représentations. Un résultat final n'est plus seulement « la division par une base a donné zéro » ; il contient une identité directe dans les générateurs initiaux, ce qui rend la preuve portable et falsifiable.

9 Réflexion sur le test décisif et la correction O76-1.1

Le test externe a changé ma lecture de la frontière réelle d'O76. Le problème n'était pas qu'un calcul de Gröbner puisse parfois être lent : cela est normal. Le problème exploitable était qu'une forme structurelle riche, reconnue par l'observation des empreintes, n'avait pas de trajectoire spécialisée

avant de tomber dans le moteur générique. La théorie des empreintes impose précisément de profiter des distinctions observables sans les confondre avec une décision de vérité. La stratégie triangulaire est donc cohérente avec la théorie : elle observe une structure, choisit une transformation adaptée, mais ne conclut que par une identité algébrique vérifiable.

Le second enseignement est négatif et essentiel. Les affirmations « Collatz » et « Riemann » ne doivent pas être transformées artificiellement en tâches propositionnelles parce que le parseur ne comprend pas leur sémantique. L'ancienne sortie `UNRESOLVED` était correcte sur le fond mais insuffisamment descriptive sur la cause. O76-1.1 répare cette frontière en créant `UNSUPPORTED_FORM` et en séparant le diagnostic de forme de la conclusion mathématique.

La troisième leçon concerne la mémoire. Une optimisation n'est admissible que si elle peut être définie indépendamment du nom d'un benchmark. La nouvelle brique ne connaît ni « chain12 », ni le hash de l'expérience : elle reconnaît une famille polynomiale et construit un certificat général. Cette propriété est un garde-fou contre le benchmark overfitting au niveau logiciel.

Dernière régression corrigée. Après les premiers tests positifs, la batterie automatisée a découvert un cas subtil : l'énoncé de Collatz contient le fragment `T(n)=n/2`, de sorte qu'un détecteur syntaxique trop faible pouvait confondre un signe d'égalité interne avec une relation mathématique entière. Cette erreur a été corrigée en exigeant le rejeu de la relation complète par le parseur polynomial. Le résultat final est désormais `UNSUPPORTED_FORM/UNRESOLVED`, ce qui aligne le diagnostic avec la sémantique réellement supportée. Enfin, l'expérience ne permet pas de conclure que O76 résout les mathématiques ouvertes. Elle montre quelque chose de plus précis : O76 peut apprendre de son expérience d'exécution au sens où le registre d'erreurs provoque une nouvelle stratégie structurelle, tout en gardant une séparation stricte entre observation, recherche et preuve. La réussite décisive est donc la combinaison « capacité accrue + refus des pseudo-preuves », et non la production artificielle d'une réponse à une conjecture ouverte.

10 Conclusion

La lecture globale des deux archives montre une idée cohérente : l'observation doit être relative, structurée et récursive, tandis que la preuve doit être exacte, traçable et séparée des statistiques. O76 tente de réunir ces deux axes sans les confondre.

La décision architecturale la plus importante est donc de ne jamais utiliser l'« émergence » comme excuse pour relâcher la rigueur. Au contraire, plus le modèle devient récursif et capable de réorganiser ses trajectoires, plus la frontière de certification doit devenir stricte. O76 est ainsi conçu comme un moteur qui peut apprendre de son expérience de calcul sans apprendre une réponse secrète, et qui peut explorer sans transformer son exploration en vérité.

Références internes

- `DDD_Base_theorie_des_empreintes_complete_order_5.tex`
- `th_emp_observation_formalisme_0.tex`
- `th_emp_suite_formalisme_order_69.tex`
- registres et audits O75 autres présents dans le corpus fourni.